

SFTP Automation Solution — Credence File Transfer

Problem Statement

Files from **ADAM** are uploaded to their own SFTP server. Credence staff were manually monitoring and transferring these files to the **Artiva SFTP**, where Artiva's systems pick them up for processing.

Current Manual Flow



Challenges

- Files are large and take variable time to upload
- No fixed file size or arrival schedule
- No access or control over ADAM's SFTP server
- Automating too early risks picking up an incomplete file
- Entirely manual process, not scalable

Why Simple Polling Won't Work

Approach	Problem
File appears on SFTP → trigger download	File may still be mid-upload

File size stability check	Not scalable, not deterministic
File age / last modified time	Slow uploads can still be in progress
Exclusive file lock check	No access to ADAM's server internals
SFTP server post-upload hook	No control over ADAM's server config

Chosen Solution — Email-Triggered Automation

ADAM sends a **notification email to Credence** after every completed upload. This email contains the filename and acts as the **single source of truth** that the file is fully uploaded and ready.

Why This Works

- The email is sent **by ADAM** after upload completes — zero race condition
 - No polling of file size or timestamps
 - No dependency on ADAM's server configuration
 - No changes required from ADAM
 - Fully event-driven and scalable
-

Automated Flow

```
ADAM uploads file to ADAM's SFTP
|
v
ADAM sends notification email to Credence Outlook inbox (contains filename)
|
v
Python script monitors Outlook inbox via exchangelib
|
v
Email detected → filename extracted from email
|
v
Script connects to ADAM's SFTP → downloads file
|
v
```

```
Script connects to Artiva SFTP → uploads file
|
v
Artiva picks up file for processing
```

Implementation Plan

1. Email Monitoring (Outlook/Exchange)

- Connect to Credence Outlook inbox using `exchangelib` Python library
- Poll for unread emails from ADAM's known sender address
- Parse subject/body to extract filename
- Mark email as read after processing to avoid duplicate triggers

2. SFTP Download from ADAM

- Use `paramiko` library to connect to ADAM's SFTP
- Download the file identified from the email
- Stage temporarily on local/intermediate server

3. SFTP Upload to Artiva

- Use `paramiko` to connect to Artiva SFTP
- Upload the downloaded file to the correct directory
- Artiva's existing pickup process runs from here unchanged

4. Logging & Alerting

- Log every transfer with timestamp, filename, and status
 - Alert on failures via email notification
 - Retain logs for audit trail
-

Tech Stack

--	--

Component	Tool
Language	Python 3.x
Email monitoring	<code>exchangelib</code> (Outlook/Exchange)
SFTP client	<code>paramiko</code>
Scheduling / service	Windows Task Scheduler or Windows Service
Logging	Python <code>logging</code> module

Local Test Environment Setup

Overview

Using **OpenSSH Server** on Windows (Amazon WorkSpace) to simulate both ADAM and Artiva SFTP servers locally for testing before going to production.

```
localhost:22 (OpenSSH Server)
|
├─ User: adam_test    → C:\sftp\adam    (mimics ADAM's SFTP)
└─ User: artiva_test → C:\sftp\artiva  (mimics Artiva's SFTP)
```

Prerequisites

- Ask IT to install **OpenSSH Server** on your Amazon WorkSpace
- Confirm installation by running in PowerShell:

```
Get-WindowsCapability -Online | Where-Object Name -like 'OpenSSH*'
```

Expected output:

```
Name   : OpenSSH.Server~~~~0.0.1.0
State  : Installed
```

Step 1 — Start OpenSSH Service

```
Start-Service sshd
Set-Service -Name sshd -StartupType Automatic
```

Step 2 — Create Folder Structure

```
New-Item -ItemType Directory -Path C:\sftp\adam  
New-Item -ItemType Directory -Path C:\sftp\artiva
```

Step 3 — Create Test Users

```
# ADAM test user  
New-LocalUser -Name "adam_test" -Password (ConvertTo-SecureString "TestPass123!")  
  
# Artiva test user  
New-LocalUser -Name "artiva_test" -Password (ConvertTo-SecureString "TestPass456!")
```

Step 4 — Configure sshd_config

Add the following to the bottom of C:\ProgramData\ssh\sshd_config :

```
Match User adam_test  
    ForceCommand internal-sftp  
    ChrootDirectory C:\sftp\adam  
    PermitTunnel no  
    AllowAgentForwarding no  
    AllowTcpForwarding no  
  
Match User artiva_test  
    ForceCommand internal-sftp  
    ChrootDirectory C:\sftp\artiva  
    PermitTunnel no  
    AllowAgentForwarding no  
    AllowTcpForwarding no
```

Step 5 — Restart OpenSSH

```
Restart-Service sshd
```

Step 6 — Verify via FileZilla Client

Field	ADAM	Artiva
Host	localhost	localhost
Port	22	22

Username	adam_test	artiva_test
Password	TestPass123!	TestPass456!
Root Folder	C:\sftp\adam	C:\sftp\artiva

Test Flow

1. Drop a test file into `C:\sftp\adam` (simulates ADAM uploading)
 2. Send a test email to your Outlook inbox with the filename in subject/body
 3. Python script detects email → extracts filename
 4. Script connects to `adam_test` SFTP → downloads file
 5. Script connects to `artiva_test` SFTP → uploads file
 6. Verify file appears in `C:\sftp\artiva`
-

Open Questions

- ADAM SFTP credentials (host, port, username, key/password)
 - Artiva SFTP credentials (host, port, username, key/password)
 - Is the filename in the email subject, body, or both?
 - Where should files be staged temporarily during transfer?
 - Who monitors alerts if a transfer fails?
-

Next Steps

- Ask IT to install OpenSSH Server on Workspace
- Set up local test environment (steps above)
- Test SFTP connections via FileZilla
- Build and test Python automation script against local test environment

- Gather production credentials for ADAM and Artiva SFTPs
 - Deploy to production and run parallel with manual process to validate
 - Decommission manual process once stable
-

Note: Test credentials (`TestPass123!` , `TestPass456!`) are for local testing only. Never use in production.